# Deadlock Detector 심층 기술 보고서

버전: 1.0.0 작성일: 2026-01-25 대상 독자: 시스템 아키텍트, 백엔드 개발자, OS 연구자

---

## 목차

1. 데드락 개념 및 조건
2. DFS 기반 사이클 탐지 알고리즘
3. 희생자 선택 전략 심층 비교
4. 롤백 메커니즘
5. 예방 vs 탐지 비교

---

## 1. 데드락 개념 및 조건

### 1.1 데드락의 정의

데드락(Deadlock)은 두 개 이상의 프로세스(또는 에이전트)가 서로가 가진 자원을 기다리며 무한히 대기하는 상태입니다. 이 상태에서는 외부 개입 없이는 어떤 프로세스도 진행할 수 없습니다.

### 1.2 데드락의 4가지 필요조건 (Coffman Conditions)

데드락이 발생하려면 다음 네 가지 조건이 동시에 만족되어야 합니다:

#### 1.2.1 상호 배제 (Mutual Exclusion)

```
정의: 자원은 한 번에 하나의 프로세스만 사용할 수 있다.

AI 에이전트 예시:
- GPU 자원: 한 에이전트만 GPU 연산 수행
- API 키: 분당 요청 제한이 있는 API
- 데이터베이스 락: 특정 레코드에 대한 배타적 접근

구현:
interface Resource {
  heldBy: string | null;  // null이면 사용 가능, 아니면 보유 에이전트 ID
}
```

#### 1.2.2 점유 대기 (Hold and Wait)

```
정의: 프로세스가 자원을 보유한 상태에서 다른 자원을 기다린다.

AI 에이전트 예시:
- Agent-1이 GPU를 보유하면서 메모리 할당을 대기
- Agent-2가 데이터셋을 로드한 상태에서 모델 가중치 파일을 대기

구현:
interface Agent {
  heldResources: string[];      // 현재 보유 자원
  waitingFor: string | null;    // 대기 중인 자원
}
```

#### 1.2.3 비선점 (No Preemption)

```
정의: 프로세스가 자원을 자발적으로 해제할 때까지 뺏을 수 없다.

AI 에이전트 예시:
- 실행 중인 추론 작업에서 GPU를 강제로 회수 불가
- 진행 중인 파일 쓰기 작업 중단 불가
```

```
구현:
// 자원 해제는 에이전트의 명시적 요청에 의해서만 수행
releaseResources(agentId: string, resourceIds: string[]): boolean
```

#### 1.2.4 순환 대기 (Circular Wait)

```
정의: 프로세스들이 원형으로 서로의 자원을 기다린다.

AI 에이전트 예시:
Agent-1 → (waits for) → Resource-A → (held by) → Agent-2
Agent-2 → (waits for) → Resource-B → (held by) → Agent-3
Agent-3 → (waits for) → Resource-C → (held by) → Agent-1

시각화:
┌─────────┐     ┌─────────┐     ┌─────────┐
│ Agent-1 │────>│ Agent-2 │────>│ Agent-3 │
└─────────┘     └─────────┘     └─────────┘
     ▲                                  │
     └──────────────────────────────────┘
```

### 1.3 데드락 처리 전략

| 전략 | 설명 | 장점 | 단점 | 본 시스템 적용 |
|---|---|---|---|---|
| 예방 (Prevention) | 4가지 조건 중 하나 제거 | 데드락 완전 방지 | 자원 활용도 저하 | 부분 적용 |
| 회피 (Avoidance) | 안전 상태 유지 | 높은 자원 활용도 | 오버헤드 | 은행원 알고리즘 |
| 탐지 (Detection) | 데드락 발생 후 감지 | 최대 자원 활용 | 복구 비용 | WFG + DFS |
| 무시 (Ignorance) | 데드락 무시 | 단순함 | 시스템 장애 | 미적용 |

---

## 2. DFS 기반 사이클 탐지 알고리즘

### 2.1 알고리즘 개요

Wait-For Graph에서 사이클을 탐지하기 위해 깊이 우선 탐색(DFS)을 사용합니다. 그래프 이론에서 방향 그래프의 사이클 존재 여부는 DFS 중 백 엣지(**Back Edge**)의 발견으로 판단할 수 있습니다.

### 2.2 Three-Color Marking Algorithm

노드의 방문 상태를 세 가지 색상으로 관리합니다:

```
enum NodeState {
  UNVISITED = 'unvisited',    // 흰색: 미방문
  VISITING = 'visiting',      // 회색: DFS 스택에 있음
  VISITED = 'visited',        // 검은색: 탐색 완료
}
```

### 2.3 알고리즘 의사코드

```
Algorithm: DetectCycles(Graph G)
Input: 방향 그래프 G = (V, E)
Output: 발견된 사이클 목록
```

```
1. for each vertex v in V:
      color[v] = WHITE
      parent[v] = NIL

2. cycles = empty list
3. path = empty stack

4. for each vertex v in V:
      if color[v] == WHITE:
         DFS-Visit(G, v, cycles, path)

5. return cycles

---

Procedure: DFS-Visit(G, u, cycles, path)

1. color[u] = GRAY
2. path.push(u)

3. for each vertex v in Adj[u]:
      if color[v] == GRAY:
         // 백 엣지 발견 - 사이클!
         cycle = ExtractCycle(path, v)
         cycles.add(cycle)

      else if color[v] == WHITE:
         parent[v] = u
         DFS-Visit(G, v, cycles, path)

4. path.pop()
5. color[u] = BLACK
```

### 2.4 구현 상세

#### CycleDetector 클래스

```
class CycleDetector {
  private graph: WaitForGraph;
  private dfsNodes: Map<string, DFSNode>;
  private cycles: DeadlockCycle[];
  private currentPath: string[];

  /**
   * DFS 방문 메서드 - 핵심 알고리즘
   */
  private dfsVisit(agentId: string): void {
    const node = this.dfsNodes.get(agentId)!;

    // 1. 현재 노드를 VISITING으로 표시
    node.state = NodeState.VISITING;
    this.currentPath.push(agentId);

    // 2. 모든 이웃 노드 탐색
    const outgoingEdges = this.getOutgoingEdges(agentId);
```

```
    for (const edge of outgoingEdges) {
      const neighborId = edge.toAgentId;
      const neighbor = this.dfsNodes.get(neighborId);

      if (neighbor.state === NodeState.VISITING) {
        // 3. 백 엣지 발견 - 사이클!
        this.extractCycle(agentId, neighborId);

      } else if (neighbor.state === NodeState.UNVISITED) {
        // 4. 트리 엣지 - 재귀 탐색
        neighbor.parent = agentId;
        this.dfsVisit(neighborId);
      }
      // VISITED 상태는 건너뜀 (크로스 엣지)
    }

    // 5. 탐색 완료
    node.state = NodeState.VISITED;
    this.currentPath.pop();
  }
}
```

사이클 추출

```
private extractCycle(fromId: string, toId: string): void {
  // currentPath에서 사이클 시작점(toId) 찾기
  const toIndex = this.currentPath.indexOf(toId);
  const fromIndex = this.currentPath.indexOf(fromId);

  if (toIndex === -1 || fromIndex === -1) return;

  // 사이클 경로 추출: [toId, ..., fromId]
  const cycleAgentIds = this.currentPath.slice(toIndex, fromIndex + 1);

  // 사이클을 형성하는 엣지 수집
  const cycleEdges: WaitForEdge[] = [];
  for (let i = 0; i < cycleAgentIds.length; i++) {
    const current = cycleAgentIds[i];
    const next = cycleAgentIds[(i + 1) % cycleAgentIds.length];
    const edge = this.findEdge(current, next);
    if (edge) cycleEdges.push(edge);
  }

  // DeadlockCycle 생성 및 저장
  const cycle = createDeadlockCycle(cycleAgentIds, cycleEdges);
  this.cycles.push(cycle);
}
```

### 2.5 복잡도 분석

| 측면 | 복잡도 | 설명 |
|---|---|---|
| 시간 복잡도 | O(V + E) | 각 노드와 엣지를 최대 한 번씩 방문 |
| 공간 복잡도 | O(V) | 색상 배열 + 재귀 스택 |
| 사이클 추출 | O(V) | 경로에서 사이클 부분 추출 |
| 전체 | O(V + E) | 선형 시간 알고리즘 |

### 2.6 에지 케이스 처리

```
// 1. 자기 자신을 가리키는 엣지 (Self-loop)
if (edge.fromAgentId === edge.toAgentId) {
  // 즉시 사이클로 처리
  this.cycles.push(createDeadlockCycle([agentId], [edge]));
}

// 2. 연결되지 않은 그래프 (Disconnected Graph)
// 모든 노드에서 DFS 시작하여 처리
for (const agentId of this.graph.agents.keys()) {
  if (node.state === NodeState.UNVISITED) {
    this.dfsVisit(agentId);  // 각 연결 요소 탐색
  }
}

// 3. 빈 그래프
if (this.graph.agents.size === 0) {
  return [];  // 빈 사이클 목록 반환
}
```

### 2.7 대안 알고리즘: Tarjan's SCC

복잡한 그래프에서 모든 강연결요소(SCC)를 찾아 사이클을 식별하는 Tarjan 알고리즘도 구현되어 있습니다:

```
class GraphBasedCycleDetector {
  private adjacencyList: Map<string, string[]>;

  detect(): DeadlockCycle[] {
    // 인접 리스트 기반 탐색
    // 멀티 그래프에서 더 효율적
  }
}
```

---

## 3. 희생자 선택 전략 심층 비교

### 3.1 전략별 알고리즘 분석

#### Lowest Priority First

```
private selectByLowestPriority(agents: Agent[]): VictimSelectionResult {
  // O(n log n) 정렬 후 첫 번째 선택
  const sorted = [...agents].sort((a, b) => a.priority - b.priority);
  const victim = sorted[0];

  return {
    victim,
    reason: `Lowest priority (${victim.priority})`,
    score: victim.priority,
  };
}
```

수학적 모델:

```
Score(a) = priority(a)
Victim = argmin{Score(a) | a ∈ Cycle}
```

### Youngest First

```
private selectByYoungest(agents: Agent[]): VictimSelectionResult {
  const sorted = [...agents].sort(
    (a, b) => b.createdAt.getTime() - a.createdAt.getTime()
  );
  const victim = sorted[0];
  const ageMs = Date.now() - victim.createdAt.getTime();

  return {
    victim,
    reason: `Youngest agent (${ageMs}ms old)`,
    score: ageMs,
  };
}
```

수학적 모델:

```
Score(a) = now - createdAt(a)
Victim = argmin{Score(a) | a ∈ Cycle}
```

### Most Resources Held

```
private selectByMostResources(agents: Agent[]): VictimSelectionResult {
  const sorted = [...agents].sort(
    (a, b) => b.heldResources.length - a.heldResources.length
  );
  const victim = sorted[0];

  return {
    victim,
    reason: `Holding most resources (${victim.heldResources.length})`,
    score: -victim.heldResources.length,  // 음수: 많을수록 우선
  };
}
```

수학적 모델:

```
Score(a) = -|heldResources(a)|
Victim = argmin{Score(a) | a ∈ Cycle}
       = argmax{|heldResources(a)| | a ∈ Cycle}
```

### Minimum Dependencies

```
private selectByMinimumDependencies(agents: Agent[]): VictimSelectionResult {
  const sorted = [...agents].sort(
    (a, b) => a.heldResources.length - b.heldResources.length
  );
  const victim = sorted[0];

  return {
    victim,
    reason: `Fewest dependencies (${victim.heldResources.length})`,
    score: victim.heldResources.length,
  };
}
```

### 3.2 성능 특성 비교

| 전략 | 시간 복잡도 | 공간 복잡도 | 결정론적 | 공정성 지수 |
|---|---|---|---|---|
| Lowest Priority | O(n log n) | O(n) | Yes | 0.2 |
| Youngest | O(n log n) | O(n) | Yes | 0.5 |
| Most Resources | O(n log n) | O(n) | Yes | 0.6 |
| Min Dependencies | O(n log n) | O(n) | Yes | 0.8 |
| Random | O(1) | O(1) | No | 1.0 |

### 3.3 합의 기반 선택 (Composite Selector)

여러 전략을 동시에 실행하고 다수결로 희생자를 결정합니다:

```
class CompositeVictimSelector {
  getConsensusVictim(cycle: DeadlockCycle, agents: Map<string, Agent>):
    VictimSelectionResult | null {

    const recommendations = this.getAllRecommendations(cycle, agents);

    // 에이전트별 득표수 집계
    const counts = new Map<string, number>();
    for (const result of recommendations.values()) {
      const count = counts.get(result.victim.id) || 0;
      counts.set(result.victim.id, count + 1);
    }

    // 최다 득표 에이전트 선택
    let maxCount = 0;
    let consensusAgentId: string | null = null;
    for (const [agentId, count] of counts) {
      if (count > maxCount) {
        maxCount = count;
        consensusAgentId = agentId;
      }
    }

    return {
      ...recommendations.get(/* first matching strategy */),
      reason: `Consensus: ${maxCount}/${recommendations.size} strategies`,
    };
  }
}
```

---

## 4. 롤백 메커니즘

### 4.1 체크포인트 아키텍처

```
graph LR
    subgraph "체크포인트 저장소"
        CP1[Checkpoint 1]
        CP2[Checkpoint 2]
        CP3[Checkpoint N]
    end

    subgraph "에이전트 상태"
```

```
        State[현재 상태<br/>heldResources<br/>waitingFor<br/>priority]
    end

    State -->|createCheckpoint| CP3
    CP2 -->|rollback| State

    style CP3 fill:#66bb6a
    style CP2 fill:#42a5f5
```

### 4.2 상태 저장 프로세스

```
public createCheckpoint(agent: Agent): Checkpoint {
  const sequenceNumber = (this.currentSequence.get(agent.id) || 0) + 1;

  const checkpoint: Checkpoint = {
    id: this.generateCheckpointId(agent.id, sequenceNumber),
    agentId: agent.id,
    heldResources: [...agent.heldResources],  // 깊은 복사
    state: agent.state,
    timestamp: new Date(),
    sequenceNumber,
  };

  // FIFO 방식 체크포인트 관리
  const checkpoints = this.checkpoints.get(agent.id) || [];
  checkpoints.push(checkpoint);

  if (checkpoints.length > this.maxCheckpointsPerAgent) {
    checkpoints.shift();  // 가장 오래된 것 삭제
  }

  this.checkpoints.set(agent.id, checkpoints);
  return checkpoint;
}
```

### 4.3 롤백 프로세스

```
private performRollback(
  agent: Agent,
  checkpoint: Checkpoint,
  resources: Map<string, Resource>
): string[] {
  const resourcesReleased: string[] = [];

  // 1. 체크포인트 이후 획득한 자원 해제
  for (const resourceId of agent.heldResources) {
    if (!checkpoint.heldResources.includes(resourceId)) {
      const resource = resources.get(resourceId);
      if (resource) {
        resource.heldBy = null;  // 자원 해제
        resourcesReleased.push(resourceId);
      }
    }
  }

  // 2. 에이전트 상태 복원
  agent.heldResources = [...checkpoint.heldResources];
  agent.state = checkpoint.state;
```

```
  agent.waitingFor = null;
  agent.updatedAt = new Date();

  return resourcesReleased;
}
```

### 4.4 데이터 일관성 보장

```
// 트랜잭션적 롤백 (원자성 보장)
public rollback(
  agentId: string,
  checkpointId: string | null,
  agents: Map<string, Agent>,
  resources: Map<string, Resource>
): RollbackResult {
  try {
    // 1. 검증
    const agent = agents.get(agentId);
    if (!agent) throw new Error('Agent not found');

    const checkpoint = this.findCheckpoint(agentId, checkpointId);
    if (!checkpoint) throw new Error('Checkpoint not found');

    // 2. 롤백 수행
    const released = this.performRollback(agent, checkpoint, resources);

    // 3. 성공 결과 반환
    return {
      success: true,
      agentId,
      checkpointId: checkpoint.id,
      resourcesReleased: released,
      timestamp: new Date(),
    };

  } catch (error) {
    // 4. 실패 시 롤백 없음 (상태 유지)
    return {
      success: false,
      agentId,
      checkpointId: checkpointId || '',
      resourcesReleased: [],
      timestamp: new Date(),
      error: error.message,
    };
  }
}
```

---

## 5. 예방 vs 탐지 비교

### 5.1 데드락 예방 (Prevention)

4가지 Coffman 조건 중 하나를 제거하여 데드락 자체가 발생하지 않도록 합니다.

| 조건 제거 방법 | 구현 방식 | 단점 |
|---|---|---|
| 상호 배제 제거 | 읽기 전용 자원만 사용 | 쓰기 작업 불가 |
| 점유 대기 제거 | 모든 자원 한 번에 요청 | 자원 활용도 저하 |
| 비선점 제거 | 강제 자원 회수 | 작업 손실 |
| 순환 대기 제거 | 자원 순서 부여 | 유연성 감소 |

### 5.2 데드락 회피 (Avoidance) - 은행원 알고리즘

```
graph TB
    Request[자원 요청] --> Check{가용 자원 충분?}
    Check -->|No| Wait[대기]
    Check -->|Yes| Simulate[할당 시뮬레이션]

    Simulate --> Safety{안전 상태?}
    Safety -->|Yes| Allocate[할당]
    Safety -->|No| Wait

    Allocate --> Done[완료]
    Wait --> Release[자원 해제 대기]
    Release --> Request

    style Allocate fill:#66bb6a
    style Wait fill:#ffa726
```

안전 상태 검사 알고리즘:

```
class SafetyChecker {
  private isSafeState(state: AllocationState): SafetyResult {
    const work = new Map(state.available);
    const finish = new Map<string, boolean>();
    const safeSequence: string[] = [];

    // 모든 에이전트를 미완료로 초기화
    for (const agentId of this.agents.keys()) {
      finish.set(agentId, false);
    }

    // 완료 가능한 에이전트 찾기
    let found = true;
    while (found) {
      found = false;
      for (const agentId of this.agents.keys()) {
        if (finish.get(agentId)) continue;

        const need = this.calculateNeed(agentId, state);
        if (this.canFinish(need, work)) {
          // 에이전트 완료 처리
          this.releaseResources(agentId, state, work);
          finish.set(agentId, true);
          safeSequence.push(agentId);
          found = true;
          break;
        }
      }
    }
```

```
    return {
      isSafe: Array.from(finish.values()).every(f => f),
      safeSequence,
      work,
      finish,
    };
  }
}
```

### 5.3 데드락 탐지 (Detection) - WFG + DFS

```
graph TB
    subgraph "주기적 탐지"
        Timer[타이머] --> Detect[사이클 탐지]
        Detect --> Found{데드락 발견?}
        Found -->|No| Timer
        Found -->|Yes| Select[희생자 선택]
        Select --> Rollback[롤백]
        Rollback --> Timer
    end

    style Found fill:#ef5350
    style Select fill:#ffa726
    style Rollback fill:#66bb6a
```

### 5.4 전략별 비교

| 측면 | 예방 | 회피 | 탐지 |
|---|---|---|---|
| 자원 활용도 | 낮음 | 중간 | 높음 |
| 오버헤드 | 낮음 | O(n^2 * m) | O(V + E) |
| 구현 복잡도 | 낮음 | 높음 | 중간 |
| 복구 비용 | 없음 | 없음 | 롤백 필요 |
| 실시간성 | 좋음 | 좋음 | 탐지 지연 |
| 적용 환경 | 정적 시스템 | 알려진 최대 요구량 | 동적 시스템 |

### 5.5 본 시스템의 하이브리드 접근

```
graph TB
    Request[자원 요청] --> Banker{은행원 알고리즘<br/>안전 상태?}

    Banker -->|Yes| Allocate[자원 할당]
    Banker -->|No| Queue[대기열 추가]

    Queue --> Periodic[주기적 탐지]
    Periodic --> WFG[Wait-For Graph]
    WFG --> DFS[DFS 사이클 탐지]

    DFS --> Deadlock{데드락?}
    Deadlock -->|No| Monitor[계속 모니터링]
    Deadlock -->|Yes| Victim[희생자 선택]

    Victim --> Rollback[롤백]
    Rollback --> Release[자원 해제]
    Release --> Allocate

    style Allocate fill:#66bb6a
```

```
    style Deadlock fill:#ef5350
    style Rollback fill:#ffa726
```

장점: 1. 은행원 알고리즘으로 대부분의 데드락 사전 차단 2. 예외적 상황은 탐지 + 회복으로 처리 3. 자원 활용도와 안전성의 균형

---

## 부록: 성능 벤치마크

| 그래프 크기 | DFS 탐지 시간 | 은행원 검사 시간 | 롤백 시간 |
|---|---|---|---|
| 10 노드 | < 1ms | < 1ms | ~5ms |
| 100 노드 | ~5ms | ~10ms | ~50ms |
| 1,000 노드 | ~50ms | ~200ms | ~500ms |
| 10,000 노드 | ~500ms | ~3s | ~5s |

---

문서 버전: 1.0.0 최종 수정: 2026-01-25 작성자: 홍익대학교 컴퓨터공학과 졸업 프로젝트 팀